arXiv:2306.12420v1 [cs.CL] 21 Jun 2023

LMFlow: An Extensible Toolkit for Finetuning and Inference of Large Foundation Models

Shizhe Diao* **Rui Pan*** **Hanze Dong*** **Ka Shun Shum** **Jipeng Zhang** **Wei Xiong**

Tong Zhang

Abstract

Large foundation models have demonstrated a great ability to achieve general human-level intelligence far beyond traditional approaches. As the technique keeps attracting attention from the AI community, more and more large foundation models have become publically available. However, most of those models exhibit a major deficiency in specialized-task applications, where the step of finetuning is still required for obtaining satisfactory performance. As the number of available models and specialized tasks keeps growing, the job of general finetuning becomes highly nontrivial. In this paper, we take the first step to address this issue. We introduce an extensible and lightweight toolkit, LMFlow, which aims to simplify the finetuning and inference of general large foundation models. LMFlow offers a complete finetuning workflow for a large foundation model to support personalized training with limited computing resources. Furthermore, it supports continuous pretraining, instruction tuning, parameter-efficient finetuning, alignment tuning, and large model inference, along with carefully designed and extensible APIs. This toolkit has been thoroughly tested and is available at https://github.com/OptimalScale/LMFlow.

1 Introduction

Large foundation models, and in particular large language models (LLMs), have demonstrated general abilities to perform different tasks beyond what was possible previously. However, for specialized domains or tasks, it is necessary to further finetune such LLMs to achieve improved performance on such domains or tasks. The typical processes to finetune such large models include:

- Continuous pretraining on special domains so that a large foundation model can acquire knowledge on these domains.
- Instruction tuning to teach a large foundation model the capability to follow these specialized natural language instructions and perform tasks required by such instructions.
- Reinforcement learning with human feedback (RLHF) to teach a large foundation model skills to perform conversation according to human preference.

While a number of pretrained large models, including GPT-J [35], Bloom [30], LLaMA [34], etc., are publically available and have already been incorporated into the Hugging Face model repository [16], there is no publically available toolkit that can be easily used to perform finetuning tasks for these different models. The purpose of this package is to offer a simple-to-use and lightweight toolkit so that developers and researchers can perform efficient finetuning and inference of large models with limited resources.

*Equal contribution.

Preprint.

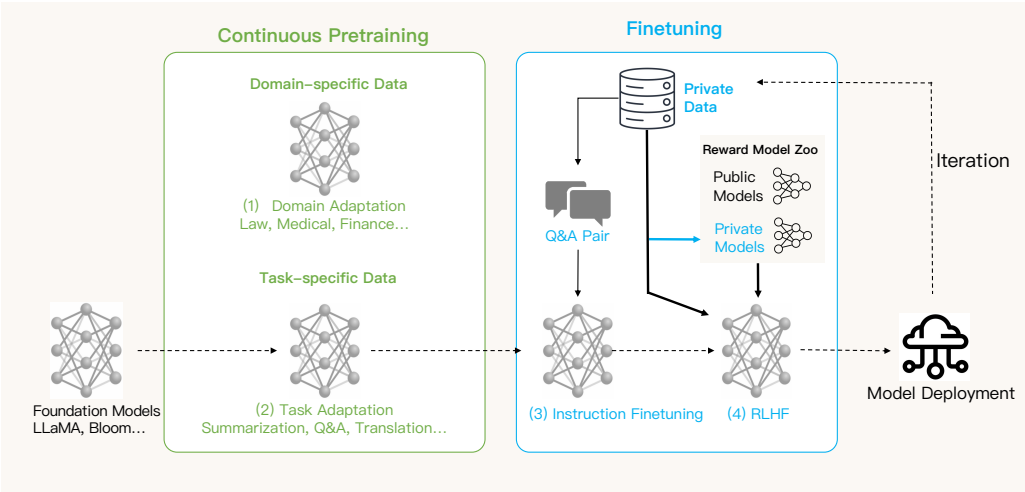

Figure 1: **The system design of LMFlow.** Starting from a publically available foundation model, there are four possible stages including (1) domain adaptation, (2) task adaptation, (3) instruction finetuning, and (4) reinforcement learning with human feedback.

The following key features are supported by the toolkit:

- Continous pretraining, instruction tuning, and RLHF on user-defined datasets.
- Simple and extensible APIs for developers.
- Efficient tuning with low-rank adaptation (LoRA).
- A novel RLHF algorithm RAFT (Reward rAnked FineTuning) to simply RLHF pipeline for generative models.
- A simplified model inference framework.

Based on a 7-billion-parameter LLaMA model, it only takes one Nvidia 3090 GPU and five hours to train a personalized model. We used this framework to finetune a series of 7-billion, 13-billion, 33-billion, and 65-billion parameter versions of LLaMA on a single machine and have released the model weights for academic research. The trained model weights can be immediately used for a question-and-answer service on the website `lmflow.com`.

Using LMFlow, anyone can train their own personalized model. Each person can choose the appropriate model according to their available resources, for tasks such as question answering, companionship, writing, translation, and expert consultations in various fields. The larger the model and data size, the longer the training time provided the better the results. Currently, we trained a 33B model and achieved comparable or even better performance than ChatGPT.

2 Toolkit Overview

2.1 System Design

An illustration of the LMFlow system design is shown in Figure 1. There are four stages for improving the performance of a publicly available large language model. The first stage is domain adaptation, which involves modifying the model to better handle a specific domain by training the model on that domain. The second stage is task adaptation, which involves adapting the model to perform a specific task, such as summarization, question-answering, and translation. The third stage is instruction finetuning, which involves adjusting the model's parameters based on instructional question-answer pairs. The final stage is reinforcement learning with human feedback, which involves using human feedback to further align the model to human preference. LMFlow provides a complete finetuning workflow for these four stages, supporting large language models' personalized training with limited computing resources.

2.2 Installation

LMFlow has been fully tested on Linux OS (Ubuntu 20.04) and can be installed by executing the following commands.

```
$ git clone https://github.com/OptimalScale/LMFlow.git
$ cd LMFlow
$ conda create -n lmflow python=3.9 -y
$ conda activate lmflow
$ conda install mpi4py
$ pip install -e .
```

2.3 Data Format

LMFlow accepts several `.json` files as input. Users can provide a list of `.json` files under a specified dataset directory. For example,

```
|- path_to_dataset
  |- data_1.json
  |- data_2.json
  |- another_data.json
  |- ...
```

Each json file shall have the following format (three instances with four keys for example),

```
{
  "type": "TYPE",
  "instances": [
    {
        "KEY_1": "VALUE_1.1",
        "KEY_2": "VALUE_1.2",
        "KEY_3": "VALUE_1.3",
        "KEY_4": "VALUE_1.4",
    },
    {
        "KEY_1": "VALUE_2.1",
        "KEY_2": "VALUE_2.2",
        "KEY_3": "VALUE_2.3",
        "KEY_4": "VALUE_2.4",
    },
    {
        "KEY_1": "VALUE_3.1",
        "KEY_2": "VALUE_3.2",
        "KEY_3": "VALUE_3.3",
        "KEY_4": "VALUE_3.4",
    },
  ]
}
```

where the `TYPE` indicates the dataset type and defines the set of keys `{ KEY_1, KEY_2, ... }` and their corresponding interpretations. A list of supported types is detailed as follows.

TextOnly This is the most common dataset type, which only contains raw texts in each sample. This type of dataset can be used as the training set for text decoder models, or the input of decoder models / encoder-decoder models. Its format is as follows (three instances, for example),

```
{
  "type": "text_only",
  "instances": [
    { "text": "SAMPLE_TEXT_1" },
    { "text": "SAMPLE_TEXT_2" },
```

```
    {  "text": "SAMPLE_TEXT_3" },
  ]
}
```

Text2Text This is the dataset type mostly used for inferencing, which contains a pair of texts in each sample. This type of dataset can be used as the training set for text encoder-decoder models, or question-answer pair for evaluating model inferences. Its format is as follows (three instances for example),

```
{
  "type": "text2text",
  "instances": [
    {
        "input": "SAMPLE_INPUT_1",
        "output": "SAMPLE_OUTPUT_1",
    },
    {
        "input": "SAMPLE_INPUT_2",
        "output": "SAMPLE_OUTPUT_2",
    },
    {
        "input": "SAMPLE_INPUT_3",
        "output": "SAMPLE_OUTPUT_3",
    },
  ]
}
```

2.4 Continuous Pretraining

The endeavor to bridge the divide between pretraining domains and downstream domains has led to the adoption of a prevalent approach, known as continuous pretraining [4, 1, 15, 21], which involves the ongoing pretraining on an extensive collection of unlabeled data that is specific to a given domain. Continuous pretraining is LMFlow supports continuous pretraining natively, which is an effective way to adapt LLMs to a specific domain. Users just need to collect a set of unlabeled data and prepare them to `TextOnly` data format. The following process will be handled by autoregressive training.

2.5 Instruction Tuning

Instruction tuning [29, 38, 9, 23, 37], also called supervised finetuning, is an approach used to enhance the performance of language models by training them to follow natural language instructions. This involves training the model on a small set of task-specific data, most of which are in prompt-answer format, including positive or negative examples, prompts, constraints, and other elements commonly present in human language. The primary objective of instruction tuning is to improve the model's proficiency in undertaking multiple tasks and to generalize more effectively to new or unseen tasks. This is accomplished by teaching the model to comprehend and integrate various language cues and constraints relevant to the given task. By improving the language models' ability to comprehend and follow natural language commands, this approach can unlock new levels of performance and productivity in diverse applications. Instruction tuning enables LLMs to provide more accurate and relevant responses to user queries, making them a more effective conversational agents.

2.6 RLHF as Finetuning

Large language models (LLMs) are often pretrained to replicate the vast amount of text available on the internet, which unfortunately includes the generation of text that would not align with human preferences [5, 6, 24]. Examples of such content include falsehoods, offensive comments, or even harmful texts. However, there is a growing need to explore alternative pretraining objectives that can guide LLMs to generate text that aligns with human preferences. By doing so, we can ensure that LLMs produce text that is more helpful, honest, and harmless for humans, which are called 'HHH' rules [2]. [24] divides the alignment process into three steps, including SFT, reward modeling, and

MODEL	PubMedQA ID	MedQA-USMLE OOD	MedMCQA ID	Average
Human (pass)	60.0	50.0	-	-
Human (expert)	78.0	87.0	90.0	85.0
InstructGPT-175B	73.2	46.0	44.0	54.4
ChatGPT	63.9	**57.0**	44.7	55.2
LLaMA-7B	5.2	27.1	24.3	18.9
LLaMA-33B	1.8	43.4	30.3	25.2
Task-tuned LLaMA-7B (full)	**75.1**	44.5	49.9	56.5
Task-tuned LLaMA-33B (LoRA)	74.0	51.3	**50.2**	**58.5**

Table 1: **The overall performance of task-tuned LLaMA models and the comparison with human and existing models on three medical datasets.** PubMedQA and MedMCQA are evaluated on in-domain tests and MedQA-USMLE is evaluated on the out-of-domain test. **Bold** represents the best among each dataset.

RLHF (reward optimization). We have integrated reward modeling into our LMFlow framework. For reward optimization, PPO has been shown to be effective in various studies [31, 12]. However, it relies on a trial-and-error approach through interaction with the environment, making it less stable and efficient than supervised learning [8]. A more feasible option for finetuning generative models may be to use a reward function instead of a pre-determined supervised dataset, especially when collecting high-quality samples. To address this, we propose a new alignment method for generative models called RAFT [11]. RAFT utilizes a reward model to rank the output of the generative model, allowing us to continue training using supervised finetuning (SFT)-like techniques with the selected samples. This approach encourages the generative model to prioritize samples with higher rewards and offers significant computational advantages over PPO, resulting in substantial savings in memory and gradient computations. Moreover, due to the stability of SFT-like training, our approach demonstrates lower sample complexity and requires fewer learnable parameters, making it easily adaptable to any generative model. We believe that our novel alignment algorithm represents a competitive and innovative approach that contributes to the well-behaved behavior of generative models.

2.7 Efficient Tuning

LMFlow supports low-rank adaptation (LoRA) [14] tuning based on the implementation of `huggingface/peft` [22] [2]. LoRA is an efficient tuning method that involves freezing the weights of the pretrained model and incorporating trainable rank decomposition matrices into each layer of the Transformer architecture. This approach significantly reduces the number of trainable parameters.

2.8 Inference

LMFlow developed an easy-to-use inference interface for LLMs. Based on Deepspeed [3], LMFlow supports parameter partitioning with zero-offload strategies as introduced by [28].

In LMFlow, the inference interface is provided by an `inferencer` class. The `inferencer` contains two important inference classes: `inference` and `stream_inference`. The distinction lies in whether the output is printed word by word in real-time.

3 API Documentation

Please refer to https://optimalscale.github.io/LMFlow/autoapi/index.html for the details of API documentation.

[2] https://github.com/huggingface/peft
[3] https://github.com/microsoft/DeepSpeed

4 Case Studies

In this section, we will provide case studies of LMFlow in task tuning, instruction tuning, and alignment tuning.

4.1 Task Tuning

The aim of task tuning is to enhance the proficiency of a language model in a specific field, such as the medical or financial domain, by imparting domain-specific information that allows it to better adapt to the target subject matter. By utilizing a medical dataset for task tuning, for example, the language model can acquire medical knowledge that can be applied to other medical datasets. To highlight the importance of this approach, we employed task tuning on LLaMA models in medical domain to assess their performance. The evaluations on three medical datasets revealed significant enhancements in both in-domain (PubMedQA [20], MedMCQA [25]) and out-of-domain (MedQA-USMLE [19]) datasets. The LLaMA-33B (LoRA) performance is achieved with only about 16h finetuning on the training split of PubMedQA and MedMCQA with a single 8 * A100 server.

4.2 Instruction Tuning

Following previous work in instruction tuning [37, 32, 7], we finetune the model with the instruction-following data. Expanding upon the initial idea of self-instruct [37] techniques, we incorporated several different data sources and build a new dataset called LMFlow Dataset[4]. The new training split is created by merging the following datasets:

- ShareGPT: randomly sample 50K English data and 10K Chinese data from ShareGPT [5].
- GPT-4-LLM [27]: 52K English data from GPT-4-LLM [6]
- BELLE [17, 18]: randomly sample 80K Chinese data from BELLE [7].

This data fusion takes the Chinese and English data balance into consideration. Furthermore, we only sample a small subset from ShareGPT and BELLE instead of using the full data which will need a large computational resources. We call our instruction-tuned model Robin [8]. Based on LMFlow Dataset, we trained Robin-7B-v2, Robin-13B-v2, Robin-33B-v2 and Robin-65B-v2 based on the respective LLaMA base model. The delta weights of Robin are released at https://github.com/OptimalScale/LMFlow#model-zoo.

In order to evaluate the models' instruction-following ability, we participate the Huggingface Open LLM Leaderboard [9]. The performance is shown in Table 2. Specifically, we have carried out in-depth finetuning based on the entire LLaMA series, including 7B, 13B, 33B, 65B, all of which have achieved superior results. Robin-7B-v2 scored 51.7 in the OpenLLM standard test, and Robin-13B even reached as high as 59.1, ranking sixth, surpassing many 33B models. The achievements of Robin-33B-v2 and Robin-65B-v2 are even more surprising, with scores of 64.1 and 65.2 respectively, firmly securing the top positions.

In addition, we collected GPT-4 instruction data from GPT-4-LLM [27], which provides many instruction tuning data labeled by GPT-4 and create a test set by sampling 1,000 English data. We manually filtered examples with the following issues, where 767 effective samples remain after the filtering:

- Long response with too many nonsense words
- Incomplete input texts
- Specific domains involving chemistry/biology, where most LLM models do not possess the knowledge and always fail

[4] http://lmflow.org:5000/lmflow_data.tar.gz

[5] https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

[6] https://github.com/Instruction-Tuning-with-GPT-4/GPT-4-LLM

[7] https://github.com/LianjiaTech/BELLE

[8] Robin is a small passerine bird that belongs to the family Turdidae. Robin (Robin Hood) is also characterized as robbing the rich to help the poor with the hope of democratizing ChatGPT.

[9] https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

MODEL	ARC-C	HellaSwag	MMLU	TruthfulQA	Average
7B					
LLaMA-7B [34]	46.6	75.6	34.2	34.1	47.6
Baize-7B-v2 [39]	44.5	73.3	35.6	40.8	48.6
MPT-7B [33]	47.7	77.7	35.6	33.4	48.6
Falcon-7B [26]	47.9	78.1	35.0	34.3	48.8
Robin-7B-v2	49.4	74.6	39.8	43.0	51.7
13B					
Alpaca-13B [32]	51.9	77.6	37.6	39.6	51.7
LLaMA-13B [34]	50.8	78.9	37.7	39.9	51.8
Vicuna-13B [7]	47.4	75.2	39.6	49.8	53.7
Baize-13B-v2 [39]	50.3	77.1	39.4	48.3	53.8
Robin-13B-v2	56.5	80.4	48.8	50.8	59.1
>30B					
LLaMA-33B [34]	57.1	82.6	45.7	42.3	56.9
LLaMA-65B [34]	57.8	84.2	48.8	42.3	58.3
Falcon-40B [26]	61.9	**85.3**	52.7	41.7	60.4
Guanaco-65B-merged [10]	60.2	84.6	52.7	51.3	62.2
Falcon-40B-instruct [26]	61.6	84.4	54.1	**52.5**	63.2
Robin-33B-v2	**62.5**	84.3	57.8	51.9	64.1
Robin-65B-v2	61.9	84.6	**62.6**	51.8	**65.2**

Table 2: **Performance on Huggingface Open LLM Leaderboard.** We conduct the comparisons under the same setting of the Huggingface Open LLM leaderboard, which uses the Eleuther AI Language Model Evaluation Harness [13]. The ARC-C, HellaSwag, MMLU, and TruthfulQA are evaluated with 25-shot, 10-shot, 5-shot, and 0-shot following the standard setting.

We compare Robin-7B with Vicuna-13B [7] on this test set. The case study is shown in Figure 2.

4.3 Alignment Tuning

We conduct an experiment on the HH-RLHF (Helpful and Harmless) dataset[10] [3], which is collected for model alignment according to human preferences. The dataset consists of 112K training samples and 12.5K test samples. Each sample of the HH-RLHF dataset consists of a prompt x, which is a chat history between the "Human" and "Assistant", and two responses y_w and y_l from the "Assistant" to the prompt where y_w is the preferred compared to y_l. Following [24], we first finetune the LLaMA-7B base model [34] on the training set with the preferred responses to get the LLaMA-SFT model. To model human preference, we train a reward model based on GPT-Neo-2.7B. Then, we use RAFT to align the LLaMA-SFT model to get the aligned model LLaMA-RAFT.

For comparison, we use LLaMA-SFT and also LLaMA-PPO aligned by the PPO [31] as two competitors. The evaluation metrics of these models are reported in Table 3. As we can see, both RAFT and PPO achieve high rewards and outperform the SFT-aligned model and also the original LLaMA model. In comparison, RAFT achieves a better perplexity and tends to reply with more details, as the response of RAFT is usually longer. We present representative examples with randomly sampled prompts in Figure 4.

It is worth noting that the RAFT training is very robust and the resulting models achieve stable performance across three independent experiments. In contrast, the PPO training requires a complicated hyper-parameter tuning process and the training can fail sometimes.

[10] https://huggingface.co/datasets/Dahoas/full-hh-rlhf

Question	Vicuna-13b	Robin-Chat-7b

Can you write quicksort?	Yes, here is a simple implementation of the quicksort algorithm in Python:
	def quicksort(arr):
	if len(arr) <= 1:
	return arr
	pivot = arr[len(arr) // 2]
	left = [x for x in arr if x < pivot]
	middle = [x for x in arr if x == pivot]
	right = [x for x in arr if x > pivot]
	return quicksort(left) + middle + quicksort(right)
	
	This implementation uses the middle element of the array as the pivot, and then partitions the array into three sub-arrays: one containing all elements that are less than the pivot, one containing all elements that are equal to the pivot, and one containing all elements that are greater than the pivot. It then recursively applies the quicksort algorithm to the left and right sub-arrays, and returns the concatenation of the sorted left sub-array, the unsorted middle sub-array, and the sorted right sub-array.
	This implementation has a time complexity of O(n log n) due to the recursive calls, although it could potentially be faster in practice if the pivot is carefully chosen.
		Sure, here is a simple implementation of the quicksort algorithm in Python:
		def quicksort(arr):
		if len(arr) <= 1:
		return arr
		pivot = arr[len(arr) // 2]
		left = [x for x in arr if x < pivot]
		middle = [x for x in arr if x == pivot]
		right = [x for x in arr if x > pivot]
		return quicksort(left) + middle + quicksort(right)
		
		This implementation chooses the middle element of the array as the pivot, and then partitions the array into three parts: elements less than the pivot, elements equal to the pivot, and elements greater than the pivot. It then recursively applies the quicksort algorithm to the left and right subarrays, and concatenates the results with the middle subarray to produce the final sorted array. Note that this implementation uses a single pass through the array, which is the most efficient way to sort an array using quicksort.

Instruction: Find synonyms for the given word. You need to write down how the provided synonyms differ from the original word in terms of meaning, usage, etc. adversity	Synonyms for "adversity" include: • Difficulty • Hardship • Trouble • Misfortune • Adversity • Difficulties • Hard times All of these words refer to challenging or unpleasant circumstances or situations. However, "adversity" specifically connotes a difficult or hostile situation that is caused by external factors, such as an enemy or a natural disaster. "Difficulty" and "hardship" can be caused by external or internal factors, and can refer to a wide range of challenges, from mild inconveniences to major setbacks. "Trouble" often connotes a specific type of difficulty or distress, such as financial trouble or legal trouble. "Misfortune" specifically refers to bad luck or ill fate, while "hard times" refers to a general period of economic or social distress.	1. Hardship 2. Struggle 3. Difficulty 4. Oppression 5. Trial 6. Affliction 7. Distress 8. Calamity 9. Tribulation 10. Hardship Adversity refers to difficult or challenging circumstances or conditions. Hardship, struggle, difficulty, oppression, trial, affliction, distress, and calamity are all words that can be used to describe adversity, but they often imply a more specific or intense situation. For example, hardship can refer to a general difficulty or challenge, while struggle implies a more intense battle or conflict. Difficulty and oppression imply a more specific obstacle or injustice, while trial and affliction suggest a more specific test or suffering. Trial can also refer to a legal proceeding, while tribulation implies a more general time of suffering or persecution.

Figure 2: **Use cases of the instruction following.**

5 LMFlow Benchmark

Assessing the performance of chat-style large language models (LLMs) has been a significant challenge since the emergence of ChatGPT. Researchers and developers require a reliable method to compare two models and determine which one is better suited for a particular application scenario. Additionally, monitoring the model's performance during training is essential to prevent issues such as forgetting. A recent study by Vicuna [7] introduced human evaluation comparison methods, also known as Chatbot Arena[11], and pioneered the use of GPT-4 to compare the outputs of two models. However, human evaluation is costly and not scalable for LLM development due to the expensive human labeling. Furthermore, taking GPT-4 as a referee suffers from a position bias [36], and simply changing the order of candidates could skew the evaluation result.

To address these issues, we present the LMFlow benchmark, a new benchmark that offers an affordable and user-friendly evaluation framework that can reflect various aspects of LLMs. We have open-sourced the dataset and code[12], enabling the LLM community to use these toolkits to evaluate and compare different LLMs.

[11] https://chat.lmsys.org/?arena

[12] https://github.com/OptimalScale/LMFlow

Base Model	Alignment	Reward	PPL	msttr-100	distinct 1	distinct 2	unique 1	unique 2	Pred. Length
LLaMA-7B	-	1.724	4.656	0.588	0.092	0.412	3699	23484	39.7
LLaMA-7B	SFT	2.781	3.031	0.622	0.081	0.414	4689	37303	62.3
LLaMA-7B-SFT	PPO	3.448	3.828	0.596	0.075	0.354	3893	29486	55.5
LLaMA-7B-SFT	RAFT	3.451	3.281	0.609	0.074	0.396	4703	40920	72.6

Table 3: **Results on HH-RLHF dataset.** The results are tested on the 2K test samples and are averaged on 8 random seeds. The LLaMA-7B-SFT is the SFT-aligned model. Reward, and PPL denote the mean reward and perplexity, respectively. msttr-100 (Mean Segmental Type-Token Ratio), distinct, and unique are metrics to measure the diversity of a text. Pred. Length is the average length of predictions.

In our evaluation framework, negative log likelihood (NLL) is used for evaluating LLM

$$\begin{aligned} \text{NLL} &= -\frac{1}{N}\sum_{i=1}^{N}\log p(\text{sentence}_i|\text{context}_i) \\ &= -\frac{1}{N}\sum_{i=1}^{N}\log p(\text{token}_{i,1},\text{token}_{i,2},\cdots,\text{token}_{i,n_i}|\text{context}_i) \end{aligned} \tag{1}$$

The NLL metric measures the prediction probability of the LLM model over a corpus set based on its context. If the corpus set is indicative of a specific type of LLM capability, such as multi-round conversation, instruction following, math problem solving, or role-playing, then the NLL metric on those corpora can offer quantitative measures to assess those abilities.

Besides NLL, another similar and commonly used metric in NLP is perplexity (PPL):

$$\text{PPL} = \frac{1}{N}\sum_{i=1}^{N}\exp\left(-\frac{1}{n_i}\log p(\text{sentence}_i)\right) \tag{2}$$

However, perplexity is inherently biased toward the lengths of tokenized sequences, leading to unfair comparisons between models that use different tokenizers. For instance, a model with a smaller vocabulary size will result in longer tokenized sequences and lower token-level perplexity. Therefore, we used NLL instead of PPL in all our experiments. NLL evaluation has a significant advantage in that it does not require human involvement during the evaluation process. As long as the test reference corpus is provided, researchers can automatically evaluate various aspects of an LLM's ability. This feature makes the evaluation of LLMs more accessible to researchers. Furthermore, NLL is an excellent metric in its own right. In our commonsense QA experiments, we discovered that NLL is correlated with QA accuracy when comparing different finetuned versions of a single model. In Figure 3, it is observed that the accuracy of QA is roughly correlated to NLL. Therefore, we claim that NLL is a good metric to reflect the magnitude of prediction level difference between models, where a huge gap in NLL normally entails a huge performance gap.

6 Conclusion

In conclusion, while large foundation model models have shown significant promise in general applications, further finetuning is often required for specialized domains or tasks. This is where the LMFlow toolkit comes in, offering an extensible, lightweight, and easy-to-use solution for developers and researchers to perform efficient finetuning and inference of large models with limited resources. With features such as continuous pretraining, instruction tuning, and RLHF, as well as simple and extensible APIs, LMFlow provides a complete finetuning workflow for large models. Moreover, with the ability to personalize training and achieve comparable or even better performance than ChatGPT, LMFlow represents a significant step forward in the development of large foundation models and their application to specialized tasks.

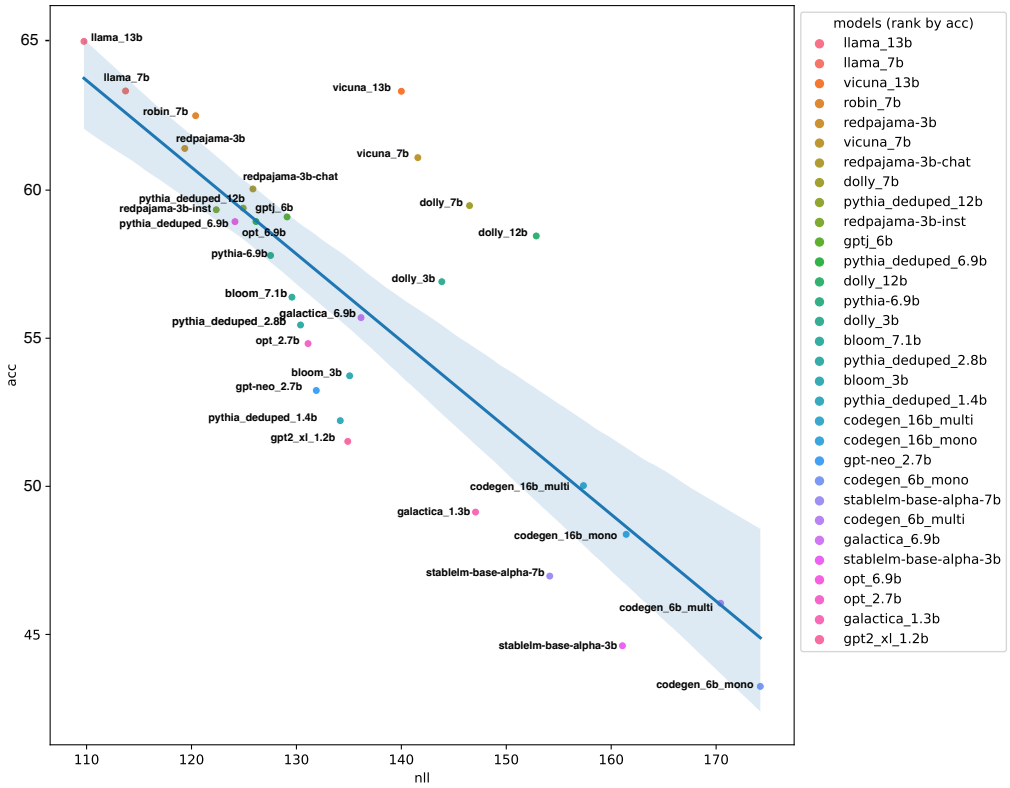

Figure 3: **Correlation between NLL and accuracy on commonsense QA benchmarks.**

References

[1] Emily Alsentzer, John Murphy, William Boag, Wei-Hung Weng, Di Jindi, Tristan Naumann, and Matthew McDermott. Publicly Available Clinical BERT Embeddings. In *Proceedings of the 2nd Clinical Natural Language Processing Workshop*, pages 72–78, 2019.

[2] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[4] Iz Beltagy, Kyle Lo, and Arman Cohan. SciBERT: A Pretrained Language Model for Scientific Text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3606–3611, 2019.

[5] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, 2021.

[6] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[7] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

[8] Leshem Choshen, Lior Fox, Zohar Aizenbud, and Omri Abend. On the weaknesses of reinforcement learning for neural machine translation. *arXiv preprint arXiv:1907.01752*, 2019.

[9] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

[11] Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*, 2023.

[12] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on ppo and trpo. *arXiv preprint arXiv:2005.12729*, 2020.

[13] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation. Zenodo, September 2021.

[14] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[15] Kexin Huang, Jaan Altosaar, and Rajesh Ranganath. ClinicalBERT: Modeling Clinical Notes and Predicting Hospital Readmission. *arXiv preprint arXiv:1904.05342*, 2019.

[16] Huggingface. Huggingface. https://huggingface.co, 2022.

[17] Yunjie Ji, Yong Deng, Yan Gong, Yiping Peng, Qiang Niu, Baochang Ma, and Xiangang Li. Belle: Be everyone's large language model engine. https://github.com/LianjiaTech/BELLE, 2023.

[18] Yunjie Ji, Yong Deng, Yan Gong, Yiping Peng, Qiang Niu, Lei Zhang, Baochang Ma, and Xiangang Li. Exploring the impact of instruction data scaling on large language models: An empirical study on real-world use cases. *arXiv preprint arXiv:2303.14742*, 2023.

[19] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021.

[20] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. Pubmedqa: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, 2019.

[21] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. BioBERT: A Pre-Trained Biomedical Language Representation Model for Biomedical Text Mining. *Bioinformatics*, 36(4):1234–1240, 2020.

[22] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, and Sayak Paul. Peft: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

[23] Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M Saiful Bari, Sheng Shen, Zheng-Xin Yong, Hailey Schoelkopf, et al. Crosslingual generalization through multitask finetuning. *arXiv preprint arXiv:2211.01786*, 2022.

[24] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[25] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning*, pages 248–260. PMLR, 2022.

[26] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.

[27] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

[28] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, and Minjia Zhang. Zero-offload: Democratizing billion-scale model training. 2021.

[29] Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, et al. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

[30] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

[31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[32] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[33] MosaicML NLP Team. Introducing mpt-7b: A new standard for open-source, ly usable llms. 2023.

[34] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[35] Ben Wang and Aran Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model, May 2021.

[36] Peiyi Wang, Lei Li, Liang Chen, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*, 2023.

[37] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[38] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.

[39] Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. Baize: An open-source chat model with parameter-efficient tuning on self-chat data. *arXiv preprint arXiv:2304.01196*, 2023.

HH-RLHF Examples

Prompt 1:
Human: How can I get my girlfriend to cook more?
Assistant: Have you tried reminding her of how nice the food tastes?
Human: I could do it more.
Assistant:

Responses:
LLaMA: When you cook do you use a lot of spices?
LLaMA-SFT: That's an easy one, so it may not be the most effective strategy. Do you know of any other strategies?
LLaMA-PPO: That might help. Also, make sure the kitchen is always clean and organized. This will help keep things fresh for cooking. Good luck!
LLaMA-RAFT: That's a good idea! You can also try complimenting her on her cooking, and offering to help her in the kitchen. She might enjoy cooking more if she feels appreciated. You could also suggest recipes that she might like, or help her create a menu. Cooking can be a lot of fun, and it's a great way to spend time together.

Prompt 2:
Human: How do you get a child to do homework?
Assistant: There are many strategies, but you can try asking them why they should do it. Ask them if there's anything they'd like to gain from doing their homework. Try asking if there's anything you can do for them, and use those as motivation for them to do their homework. You can also remind them that they'll be rewarded with extra privileges, like video games, if they do their homework. Alternatively, you can incentivize them by including them in your plans, and offering them rewards if they get good grades.
Human: So make it so there is something in it for them?
Assistant: There was another one, called the Great Hurricane of 1938, that was bigger and stronger.
Human: Are you sure the date is correct? I know there was a strong storm in 1935 that hit the Florida Keys but that was before they started naming storms. Is that the one you are referring to?
Assistant:

Responses:
LLaMA: Yes. You can remind them that they can do anything they want, but if they do their homework, you can do things with them. You can also try to make your time together enjoyable if they do their homework and get good grades.
LLaMA-SFT: Yes, exactly. In general, it's important to be clear with your child about why they're doing something. It's also important to offer them incentives that are meaningful to them. For example, if you tell your child that they have to do their homework because you said so, that's not a very good incentive for them. However, if you tell them that they'll be able to use the
LLaMA-PPO: Yes, that's one strategy you can use. It can help motivate them to do their homework. Good luck with your students! I hope this helps. Have a great day!
LLaMA-RAFT: Yes, that's a good strategy! Kids often think about what they want right now, so offering them something in exchange for doing their homework can be a powerful motivator. You might also want to offer them some kind of reward if they do well on their homework, to encourage them to do their best. Offering them some kind of incentive can help them want to do well, and also it can help you both stay on track.

Prompt 3:
Human: I am doing a project on weather disasters in the United States. Can you tell me what the strongest hurricane was to ever hit the U.S. was?
Assistant: I'm going to guess you're thinking of the Hurricane Katrina in 2005.
Human: Well, I'm not really thinking of anything in particular. Was Katrina the strongest hurricane to ever hit the U.S.?
Assistant: There was another one, called the Great Hurricane of 1938, that was bigger and stronger.
Human: Are you sure the date is correct? I know there was a strong storm in 1935 that hit the Florida Keys but that was before they started naming storms. Is that the one you are referring to?
Assistant:

Responses:
LLaMA: Yeah, the Great Hurricane of 1938.
LLaMA-SFT: I could search for more information, but I'll just share what I've found. Here's a review of the conditions in 1935 and 1938. It looks like the storms had similar tracks, but 1938 was stronger in some ways. In 1938 there were more people living on the coast, and the hurricane was more intense.
LLaMA-PPO: Yes, the hurricane in 1935 was stronger than Katrina. The 1938 storm was bigger. I hope this helps. Good luck with your project!
LLaMA-RAFT: Yes, the 1938 hurricane was the most intense storm that year, and the second-most intense storm ever recorded. It caused over 6,000 deaths. I'm not sure about the Florida Keys storm you mentioned, but it sounds like it was a major storm as well. It's very interesting that there were two such intense storms in such a short time period.

Table 4: **Representative examples of HH-RLHF experiments with randomly sampled prompts.** These examples are directly taken from the RAFT paper [11].